CL-2001: Data Structures Lab # 12

Problem 1:

Provide a C++ implementation of AVL tree which must include

- Recursive Height
- Finding Balancing Factor
- RR_Rotation(node* Parent)
- LL_Rotation(node* Parent)
- RL_Rotation(node* Parent)
- LR_Rotation(node* Parent)
- Apply on BST deletion
- Apply on BST insertion
- Display Nodes
- Test Your Code

The driver code is mentioned below

```
/*
 * C++ program to Implement AVL Tree
 */

/*
 * Node Declaration
 */
struct avl_node
{
    int data;
    struct avl_node* left;
    struct avl_node* right;
}*root;

/*
 * Class Declaration
 */
class avlTree
{
public:
    int height(avl_node*);
    int diff(avl_node*);
    avl_node* rr_rotation(avl_node*);
    avl_node* ll_rotation(avl_node*);
    avl_node* lr_rotation(avl_node*);
    avl_node* rl_rotation(avl_node*);
    avl_node* balance(avl_node*);
    avl_node* insert(avl_node*, int);
    void display(avl_node*, int);
    avl_node* FindMin(avl_node* node);
    avl_node* del(avl_node* node, int
    data);

    void find(avl_node* root, int key);
    void inorder(avl_node*);
    void preorder(avl_node*);
    void postorder(avl_node*);
    avlTree()
    {
        root = NULL;
    }
};

/*
```

```
 * Height of AVL Tree
 */
int avlTree::height(avl_node* temp)
{
    return h;
}
   // Height Difference
int avlTree::diff(avl_node* temp)
{
    return b_factor;
}
 //Right- Right Rotation
avl_node* avlTree::rr_rotation(avl_node* parent)
{
}

/*
 * Left- Left Rotation
 */
avl_node* avlTree::ll_rotation(avl_node* parent)
{
}

 //Left - Right Rotation
avl_node* avlTree::lr_rotation(avl_node* parent)
{

}

/*
 * Right- Left Rotation
 */
avl_node* avlTree::rl_rotation(avl_node* parent)
{

}

/*
 * Balancing AVL Tree
 */
avl_node* avlTree::balance(avl_node* temp)
{

}
avl_node* avlTree::FindMin(avl_node* node)
{

}

avl_node* avlTree::del(avl_node* root, int data)
{
    return node;
}
```

CL-2001: Data Structures Lab # 12

```
 * Display AVL Tree
 */
void avlTree::display(avl_node* ptr, int level)
{
}

void avlTree::find(avl_node* root, int key)
{
}

/*
 * Inorder Traversal of AVL Tree
 */
void avlTree::inorder(avl_node* tree)
{
}
/*
 * Preorder Traversal of AVL Tree
 */
void avlTree::preorder(avl_node* tree)
{
}
/*
 * Postorder Traversal of AVL Tree
 */
void avlTree::postorder(avl_node* tree)
{
}

/*
 * Main Contains Menu
 */
int main()
{
    int choice, item;
    avlTree avl;
    while (1)
    {
        cout << "\n---------------------" << endl;
        cout << "AVL Tree Implementation" << endl;
        cout << "\n---------------------" << endl;
        cout << "1.Insert Element into the tree" << endl;
        cout << "2.Insert Element into the tree" << endl;
        cout << "3.Display Balanced AVL Tree" << endl;
        cout << "4.InOrder traversal" << endl;
        cout << "5.PreOrder traversal" << endl;
        cout << "6.PostOrder traversal" << endl;
        cout << "7.Exit" << endl;
        cout << "Enter your Choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
            cout << "Enter value to be inserted: ";
            cin >> item;
            root = avl.insert(root, item);
            break;
        case 2:
            cout << "Enter value to be deleted: ";
            cin >> item;
            root = avl.del(root, item);
            break;
```

```
        case 3:
            if (root == NULL)
            {
                cout << "Tree is Empty" << endl;
                continue;
            }
            cout << "Balanced AVL Tree:" << endl;
            avl.display(root, 1);
            break;
        case 4:
            cout << "Inorder Traversal:" << endl;
            avl.inorder(root);
            cout << endl;
            break;
        case 5:
            cout << "Preorder Traversal:" << endl;
            avl.preorder(root);
            cout << endl;
            break;
        case 6:
            cout << "Postorder Traversal:" << endl;
            avl.postorder(root);
            cout << endl;
            break;
        case 7:
            exit(1);
            break;
        default:
            cout << "Wrong Choice" << endl;
        }
    }
    return 0;
}
```

Problem 2:

Write a recursive function to check if the provided tree is a valid AVL tree or not.

Problem: 3| Binary heap

a. **getMini():** It returns the root element of Min Heap.
b. **extractMin():** Removes the minimum element from MinHeap..
c. **insert():** Inserting a new key. We add a new key at the end of the tree. If new key is greater than its parent, then we don't need to do anything. Otherwise, we need to traverse up to fix the violated heap property.

Problem: 4 | Heap

A company which is responsible for the planning and construction of residential buildings in the city of Urbanville. The city has recently seen a spike in population, and there's a need for more residential buildings. The city has a specific policy about the construction of buildings; the building with the highest number of floors gets priority for construction.
A company has decided to use a heap data structure to efficiently handle this situation, where each

building's priority is determined by the number of floors it has. Ashok has the following operations he can perform:

addBuilding: This function will take the building's ID and the number of floors in that building as input and adds it to the construction queue.

constructBuilding: This function will construct the building with the highest priority (the building with the highest number of floors) and remove it from the queue.

getHighestPriorityBuilding: This function will return the ID of the building with the highest number of floors in the queue.

Problem: 5 | Convert min Heap to max Heap

Given array representation of min Heap, convert it to max Heap. **Example:**

```
Input: arr[] = [3 5 9 6 8 20 10 12 18 9]
         3
      /     \
     5       9
   /   \    /  \
  6     8  20   10
 /  \   /
12   18 9
Output: arr[] = [20 18 10 12 9 9 3 5 6 8] OR
       [any Max Heap formed from input elements]



         20
       /    \
     18      10
   /    \    /  \
  12     9  9    3
 /  \   /
5    6 8
```